🔹 UNIT 3 – Relational Design & Normalization

1. Concept Mind Map (Text Form)

Unit 3 → Relational DB Design

- → Functional Dependency (FD)
  - X → Y : X uniquely determines Y
  - Use: find keys, design schema, normalization
- → Normalization
  - Goals: remove redundancy + anomalies (insert/update/delete)
  - 1NF → atomic attributes
  - 2NF → 1NF + no **partial** dependency
  - 3NF → 2NF + no **transitive** dependency
  - BCNF → for all X→Y, **X must be superkey**
- → Anomalies
  - Insert / Update / Delete anomalies
  - Fixed by decomposition into higher NFs
- → Decomposition
  - Desirable properties:
    - Lossless-join
    - Dependency preservation
    - Minimal redundancy
- → Integrity Constraints
  - Domain (valid value range/type)
  - Entity (PK not null, unique)
  - Referential (FK must match PK)
  - Enterprise constraints (business rules)
- → Codd's Rules (important named rules)
  - Guaranteed access
  - Comprehensive data sub-language
  - Physical & logical independence

    - Integrity independence
    - Systematic NULL treatment

---

2. Super-Short Theory Points

Functional Dependency

- X → Y holds if two rows with same X always have same Y.
- Used to:
    - Find candidate keys
    - Decide decompositions / normal forms

Normal Forms – One-Liner Memory

- **1NF**: No repeating groups or multivalued attributes.
- **2NF**: 1NF + no partial FD of non-key attr on part of composite key.
- **3NF**: 2NF + no transitive FD (non-key → non-key).
    - Alternative: For every X→Y, X is superkey OR Y is prime.
- **BCNF**: For every X→Y, X is **only** superkey (no exception).

Anomalies

- **Insert**: Can't insert data because some other attribute is missing.
- **Update**: Same info repeated → must update everywhere.
- **Delete**: Deleting one row causes loss of important info.

Decomposition – Lossless Test (2 tables)

Decompose R into R1, R2:

- Let common attributes = X
- If X → R1 or X → R2 (X is key of at least one), **lossless**.

---

3. Exam "Procedure" Templates

Check 3NF of relation (template)

1. Identify **candidate key(s)**.
2. List all FDs.
3. For each FD X→Y, check:

- Is X a superkey?
- If not, is Y prime?
- If neither → 3NF violated.

4. Decompose along “bad” FDs:
    - Create relation (X ∪ Y)
    - Remove Y from original
5. Repeat till all FDs satisfy 3NF condition.

🔹 UNIT 4 – Transactions, Concurrency & Recovery

1. Concept Mind Map (Text Form)

Unit 4 → Transaction Management

- → Transaction & ACID
  - Atomicity, Consistency, Isolation, Durability
  - Transaction states: Active → Partially committed → Committed / Failed → Aborted → Terminated
- → Schedules & Serializability
  - Conflict serializability
    - Conflicting ops: R/W on same item from different Txn with at least one W
    - Precedence graph: cycle → not serializable
  - View serializability
    - Initial read, who-wrote-what, final write
- → Concurrency Control
  - Locking
    - S-lock (read), X-lock (read+write)
  - 2PL (Two-Phase Locking)
    - Growing phase (acquire), shrinking phase (release)
    - Strict 2PL: hold X locks till commit
    - Rigorous 2PL: hold S+X till commit
  - Deadlock
    - Wait-for graph, cycle = deadlock
    - Prevention (ordering, wait-die, wound-wait), detection & recovery
  - Timestamp Protocol
    - TS(T), R_TS(Q), W_TS(Q)
    - Read/Write allowed or abort based on TS comparison
- → Schedules Types
  - Recoverable, Cascadeless, Strict
- → Recovery

- Log-based recovery
    - Undo/Redo, checkpoints
- Shadow paging

---

2. Super-Short Theory Points

Conflict Serializability (check steps)

1. Nodes = transactions.
2. For each conflict pair (Ti op before Tj op), add edge Ti → Tj.
3. If graph **acyclic**, schedule is conflict serializable.

View vs Conflict

- Conflict serializable ⊂ View serializable.
- Conflict is easier to check, used in DBMS.

2PL Variants

- **Basic 2PL**: no correctness for cascading, but ensures conflict-serializable.
- **Strict 2PL**: X locks till commit → avoids cascading rollback.
- **Rigorous 2PL**: S+X locks till commit → gives strict schedules.

Deadlock Essentials

- Caused by mutual wait for locks; wait-for graph cycle.
- Recover: choose victim, rollback, release locks.

Timestamp Protocol – One-Liner

- **Read(Q) by Ti**:
    - if W_TS(Q) > TS(Ti) → abort Ti
    - else allow; R_TS(Q) = max(R_TS(Q), TS(Ti))
- **Write(Q) by Ti**:
    - if R_TS(Q) > TS(Ti) or W_TS(Q) > TS(Ti) → abort Ti
    - else allow; W_TS(Q) = TS(Ti)

Log-based Recovery

- Write-ahead log (WAL): log written *before* DB update.
- On crash:

- Undo uncommitted transactions (use old values).
- Redo committed transactions (use new values).

---

3. Quick Classification

- **Recoverable schedule**: Tj reads Ti; Tj commits only after Ti commits.
- **Cascadeless**: never read uncommitted data → no cascading abort.
- **Strict**: no read/write of item written by uncommitted Txn → simplest recovery.

🔹 UNIT 5 – NoSQL, CAP, Distributed DB

1. Concept Mind Map (Text Form)

Unit 5 → Beyond Relational DB

- → Data Types
  - Structured
  - Semi-structured (XML, JSON)
  - Unstructured (media, free text)
- → Distributed DB
  - Homogeneous / Heterogeneous
  - Architectures: Client–Server, Peer-to-peer, Federated
- → CAP Theorem
  - Consistency, Availability, Partition tolerance
  - Trade-offs: CP vs AP
- → BASE Properties
  - Basically Available
  - Soft State
  - Eventual Consistency
- → NoSQL Types
  - Key-value store
  - Document store
  - Column (wide-column)
  - Graph DB
- → SQL vs NoSQL
  - ACID vs BASE, schema, scaling
- → MongoDB Basics
  - CRUD: insert, find, update, delete
  - Aggregation pipeline
  - MapReduce

---

2. High-Yield Definitions

CAP

- **C**: all nodes see same data at same time.
- **A**: every request gets response (not necessarily latest).
- **P**: continues despite network partition.
- During partition you must choose **C or A** (can't keep both).

BASE

- **Basically Available**: system remains responsive.
- **Soft State**: state may change over time even without new input.
- **Eventual Consistency**: all replicas converge to same value eventually.

Soft state relies on eventual consistency to ensure temporary inconsistency eventually resolves.

NoSQL Types – One-Liners

- **Key-value**: (key → blob/value), super-fast lookup, caching/sessions.
- **Document**: JSON-like docs, flexible schema, e.g. MongoDB.
- **Column**: column families, wide sparse tables, e.g. Cassandra.
- **Graph**: nodes & edges, relationships, e.g. social networks (Neo4j).

Distributed DB Architectures

- **Client–Server**: central DB server, clients send queries.
- **Peer-to-peer**: all nodes are equal; each can act as client & server.
- **Federated**: autonomous heterogeneous DBs with a virtual integration layer.

---

3. MongoDB Cheat Sheet (CRUD + Agg + MapReduce)

- **Insert**:
  db.col.insertOne({...}) / insertMany([...])
- **Read**:
  db.col.find(query) / findOne(query)
- **Update**:
  db.col.updateOne(filter, {$set:{...}})
  db.col.updateMany(...)

- **Delete**:
  db.col.deleteOne(filter)
  db.col.deleteMany(filter)
- **Aggregation Pipeline**:

```
db.col.aggregate([
  { $match: {...} },
  { $group: { _id: "$field", total: { $sum: "$amt" } } },
  { $sort: { total: -1 } }
])
```

- **MapReduce** (conceptually):
  map: emit(key, value); reduce: combine values for each key.

🔹 UNIT 6 – Advanced DBMS (Active, ORDBMS, XML, JSON, etc.)

1. Concept Mind Map (Text Form)

Unit 6 → Advanced Databases

- → Active Database
  - ECA rules: Event–Condition–Action
  - Triggers
- → Deductive Database
  - Facts + Rules + Inference engine
- → Main Memory Database
  - Data in RAM, very fast, volatile
- → Object–Relational DB (ORDBMS)
  - Relational + OO features (UDT, inheritance)
  - Table inheritance (single, class, concrete)
- → Complex/Semi-structured Data
  - Semi-structured data (XML/JSON)
  - Features: flexible schema, tags, hierarchical
- → XML
  - Self-describing, markup-based, data transport
- → JSON
  - Lightweight, web APIs, data types (string, number, boolean, null, object, array)
- → Spatial Data
  - Geometric vs Geographic data

---

2. Short Concept Points

Active DB

- Uses triggers to automatically run actions on events (e.g., INSERT/UPDATE).
- ECA rule: **When E occurs, if C holds, perform A.**

Deductive DB

- Stores **facts** (“A is a student”), **rules** (“if A is student and marks > 40 then A passes”).
- Can infer new facts using logical reasoning (Datalog).

Main Memory DB

- All data in **RAM** → very high performance.
- Volatile, needs backup/logging for durability.
- Good for: caching, session data, real-time trading, etc.

ORDBMS & Table Inheritance

- ORDBMS = RDBMS + OO features.
- **Advantages**:
    - Store complex objects.
    - Less impedance mismatch with OO languages.
- **Table inheritance**:
    - Single table (one big table, nullable columns).
    - Class table (parent + child tables linked by FK).
    - Concrete table (full attributes in each child; no parent table).

Semi-structured Data & Features

- Semi-structured = not strict rows/columns; uses **tags/labels**.
- Examples: XML, JSON, HTML, emails.
- Features:
    - Self-describing
    - Varying attributes per record
    - Hierarchical & nested
    - Flexible schema

XML – When & Why

- **XML** = text format with custom tags, for data storage/transport.
- Use when:

- Need platform-independent, self-describing data.
- Data is hierarchical.
- Systems are heterogeneous and exchange data via text.

JSON – Data Types & Use

- **Why used**:
  - Lightweight, human-readable, perfect for web APIs.
- **Main data types**:
  - String: "name": "Shubham"
  - Number: "age": 20
  - Boolean: "isStudent": true
  - Null: "middleName": null
  - Object: "addr": { "city": "Pune" }
  - Array: "marks": [80, 90, 75]

Geometric vs Geographic

- **Geometric**: shapes (points, lines, polygons) – used in CAD/graphics.
- **Geographic**: real-world locations with lat/long – used in GIS, maps.

---

3. Quick "What to Write if Question Comes" Hints

- **Active vs Deductive DB**:
  - Active = triggers & rules reacting to events.
  - Deductive = facts & rules, infer new facts via logic.
- **Significance of XML/JSON**:
  - XML: rich, verbose, markup, good for document-like & hierarchical data.
  - JSON: simpler, common in REST APIs, JS-friendly, lighter than XML.
- **Explain JSON data types**:
  - List 6 types + 1 example each; mention all are text-based and language independent.